CONTRACTS · FAILURE · 1/10

Failure is part of the interface.

The error contract — the half of your API nobody writes and everybody pays for.

10 slides · swipe

Anton Petnitsky · Python / agents / backend

Error handling is not try/except.

An interface is complete only when both sides know what failure looks like. Status codes nobody defines, exceptions nobody documents, 'unknown error' — that's not an interface, that's a dare.

**If the caller can't predict failure,
the contract doesn't exist.**

The five failure cases worth writing down.

01 Invalid input — which fields, what shape of error

02 Timeout — retryable or not, and after how long

03 Partial success — which half persisted, which didn't

04 Unknown state — what the client should assume

05 Rate limit / capacity — how to back off, per whom

Every unwritten case is tomorrow's 3 a.m. archaeology dig.

What my error responses actually look like.

```
$ POST /orders

HTTP 422 { "error": "invalid_address",

"field": "street",

"retryable": false }
```

Caller reads four fields and knows exactly what to do.

What undocumented errors cost out loud.

Clients catch everything. Retry storms hit healthy servers. A partial write corrupts two systems. Support finds out from the user. The error was 'handled' — with zero information.

Catch-all handlers are where incidents collect in sample jars.

Anton Petnitsky · Python / agents / backend

Two error philosophies.

DEFENSIVE MESS

Catch everywhere

except: pass
sleep+retryblindly
no reason given

HONEST CONTRACT

Typed failures

error codes
retryable flag
named fields

Clients can retry smart only when errors are honest.

Test the contract, not just the code.

01 Contract tests per error case — real HTTP, real payloads



02 Simulate timeouts mid-write — verify partial-state reporting



03 Fuzz the boundary — what shape breaks the handler?



04 Freeze the schema — any drift breaks the build, not the client

If the caller can't rely on the shape, the shape is noise.

Anton Petnitsky · Python / agents / backend

Docs at the boundary.

Error table with codes, retryable flags, and one example each — the single highest-ROI page in the whole API documentation. Clients copy it word for word.

Error docs are read 100x more than happy-path tutorials.

The interface law.

An API is not complete until its errors are documented, typed and testable — the success path is just the opening scene.

The unhappy path is the product's real UI.

Anton Petnitsky · Python / agents / backend

CONTRACTS · KEEP THIS · 10/10

Save this for your next API review.

When someone says 'just handle the error generally', hand them this list. Vague failure handling is the most expensive kindness in software.

Save for later review

Anton Petnitsky · Python / agents / backend